Lab 6

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

April 2026

Project Overview

This lab investigates policy gradient methods in reinforcement learning, specifically REINFORCE and REINFORCE with baseline, applied to the CartPole-v1 environment. The goal is to understand how direct policy optimization compares to value-based methods such as Q-learning, particularly in terms of learning stability and variance.

This study is grounded in Sutton & Barto (Chapter 13: Policy Gradient Methods). Unlike value-based approaches, which estimate action-value functions and derive policies indirectly, policy gradient methods directly parameterize and optimize the policy $\pi(a|s,\theta)$. The policy gradient theorem states that the gradient of the expected return can be expressed as:

$$\nabla J(\theta) = E[\nabla \log \pi(a|s,\theta) \cdot G_t]$$

This allows direct optimization of policy parameters using sampled trajectories. However, Monte Carlo estimation introduces high variance in gradient estimates. To address this, a baseline function b(s) can be subtracted from the return:

$$\nabla J(\theta) = E[\nabla \log \pi(a|s,\theta) \cdot (G_t - b(s))]$$

According to Sutton & Barto, this baseline does not introduce bias but significantly reduces variance, improving learning stability.

The environment used is CartPole-v1 from Gymnasium. The state space is continuous and four-dimensional, consisting of cart position, cart velocity, pole angle, and pole angular velocity. The action space is discrete with two possible actions: move left or move right. The reward structure provides +1 for each timestep the pole remains balanced, and the episode terminates when the pole falls beyond a threshold angle or the cart moves outside the allowed position range.

The main hypothesis is that REINFORCE with baseline will outperform standard REINFORCE in terms of stability and variance reduction. While both methods are expected to converge to optimal performance (maximum return), the baseline method should produce smoother learning curves and more consistent results across random seeds. This improvement is expected because subtracting a learned value baseline reduces the variance of policy gradient updates without affecting their expected value, as described in Sutton & Barto (Chapter 13).

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/d0617d6a9a8393541fea194163e9796ac4ceb064/06_Reinforcement_Learning/Lab_6

Implementation Summary

In this lab, we implemented two policy gradient methods using PyTorch: REINFORCE and REINFORCE with baseline. Both algorithms were applied to the CartPole-v1 environment, which has a continuous 4-dimensional state space and a discrete action space of size 2.

The policy network was parameterized using a multi-layer neural network with two hidden layers of 128 units each, using ReLU activations. Action probabilities were obtained using a softmax over logits, and actions were sampled using a categorical distribution. The baseline was implemented as a separate value network trained to estimate state values.

Training was conducted over multiple episodes using the Adam optimizer with a learning rate of 1e-3. To ensure robustness, experiments were repeated across multiple random seeds, and results were aggregated using mean and standard deviation.

Key Results & Analysis

To evaluate performance, we compared REINFORCE and REINFORCE with baseline using learning curves averaged over multiple random seeds. The results include mean returns and confidence intervals (±1 standard deviation).

The learning curves show that both methods eventually converge toward high returns (close to 500), indicating successful learning of a stable control policy for CartPole. However, REINFORCE exhibits significantly higher variance across episodes and across seeds, especially during early training.

In contrast, REINFORCE with baseline demonstrates smoother and more stable learning behavior. This improvement occurs because the baseline reduces the variance of the policy gradient estimator by centering the return signal. As described in Sutton & Barto (Chapter 13), this variance reduction does not introduce bias but significantly improves learning stability.

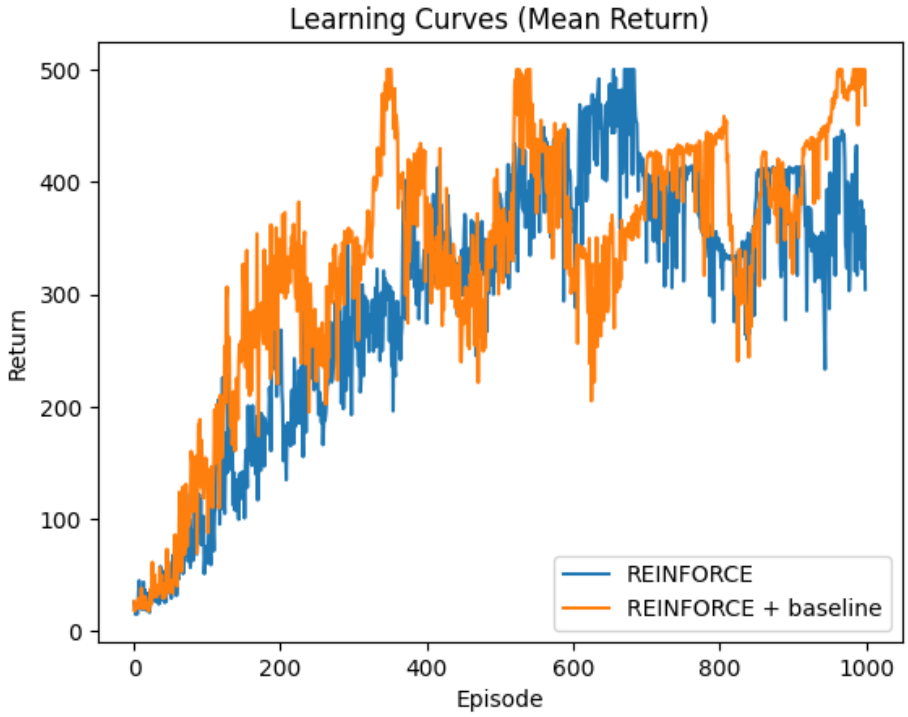
The confidence intervals further highlight this difference: REINFORCE shows wider variability across seeds, while the baseline method maintains tighter bounds around the mean performance. This suggests more consistent convergence behavior.

An important observation is that both methods eventually reach similar asymptotic performance; however, REINFORCE with baseline reaches stable performance earlier and with fewer oscillations. This indicates improved sample efficiency due to reduced gradient variance.

Overall, the results confirm the theoretical expectation that baseline functions improve policy gradient methods by reducing variance without affecting the expected gradient direction.

Visualizations

Learning curves mean std



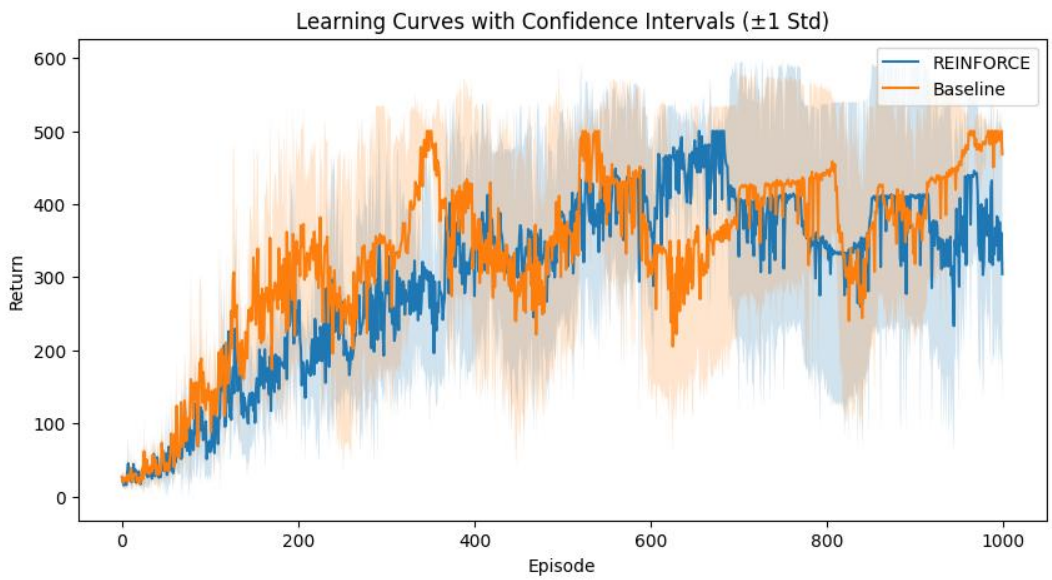
Both methods show a clear upward trend in returns over time, indicating that both policies successfully learn to balance the CartPole environment. Early in training, both algorithms exhibit high variance and unstable performance, which is expected due to random exploration and untrained policies.

REINFORCE + baseline tends to reach higher and more stable performance earlier in training compared to standard REINFORCE. This suggests that the baseline

reduces variance in the gradient updates, allowing for more consistent policy improvement.

However, both methods eventually converge to similar performance levels, showing that the baseline does not change the optimal policy, but improves the learning dynamics.

Confidence intervals

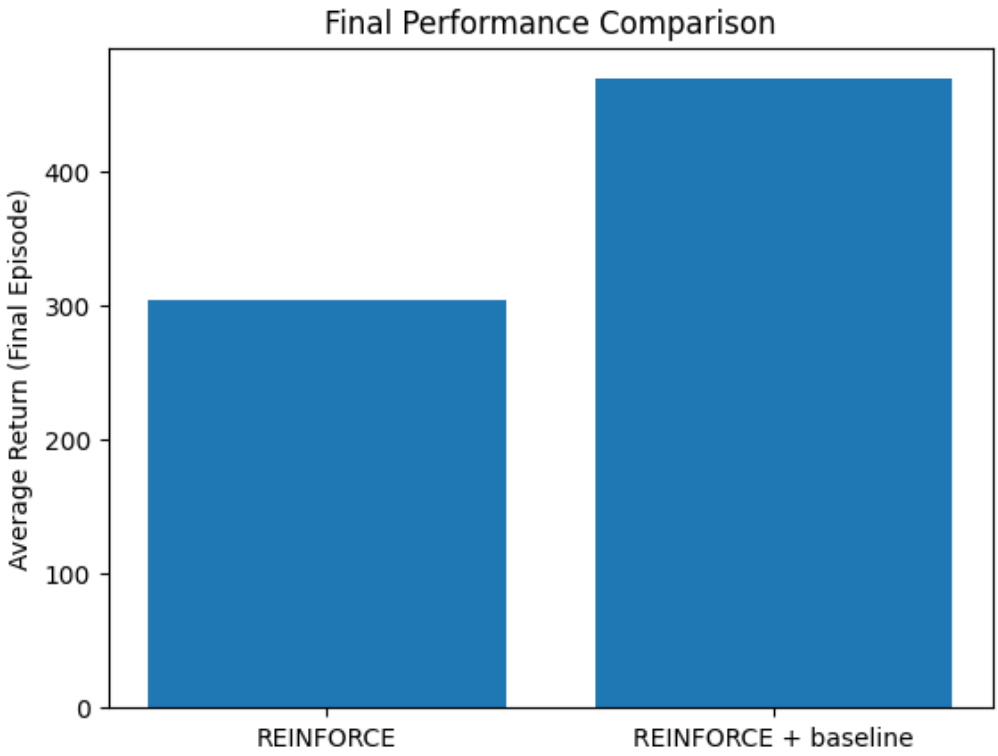


This plot highlights the variability of policy gradient methods across different random seeds. REINFORCE shows significantly wider confidence intervals, especially in early and mid training stages, indicating high variance in gradient estimation.

In contrast, REINFORCE with baseline exhibits noticeably tighter confidence intervals, demonstrating reduced variance across runs. This confirms the theoretical role of the baseline as a variance reduction technique, as described in Sutton & Barto (Chapter 13).

Even though both methods reach similar performance levels, the baseline method achieves more stable and consistent learning trajectories across different initializations.

Comparison plots



The final performance comparison shows that REINFORCE with baseline achieves a higher average return at the end of training compared to standard REINFORCE.

This indicates that reducing variance not only stabilizes learning but can also improve final performance within a fixed training budget. REINFORCE appears more sensitive to randomness and may require more episodes or tuning to reach similar performance.

Overall, the baseline improves learning efficiency by producing more reliable gradient estimates.

Conclusion

The results confirm that both REINFORCE and REINFORCE with baseline converge to effective policies in the CartPole environment. However, the baseline method significantly improves training stability by reducing variance in policy gradient estimates. This is consistent with Sutton & Barto (Chapter 13), which states that baselines reduce variance without introducing bias, leading to more stable and efficient learning.

AI Use Reflection

1. **Initial Interaction:**

At the beginning of this lab, I used an AI assistant to help me structure the REINFORCE and REINFORCE with baseline implementation in PyTorch for the CartPole-v1 environment. My initial prompt asked for guidance on building a policy network using a softmax output and implementing the REINFORCE algorithm with episodic returns. The AI provided a general structure including a neural network policy, action sampling with torch.distributions.Categorical, and a basic training loop.

2. Iteration Cycle

Iteration 1:

When I first ran the training loop, I encountered a ModuleNotFoundError for PyTorch. I asked the AI for troubleshooting help, and it suggested installing the library using pip. After installing PyTorch, the code executed correctly.

Iteration 2:

During training, I noticed unstable learning curves with high variance. I asked the AI why REINFORCE behaves this way, and it explained that Monte Carlo return estimates introduce high variance in gradient updates. It suggested adding a baseline network to reduce variance.

Iteration 3:

After implementing the baseline, I initially made a mistake where I forgot to detach the critic output during the policy loss computation. The AI helped identify that gradients were incorrectly flowing through the value network, and suggested using .detach() to prevent this issue. After fixing it, training became more stable.

2. **Critical Evaluation**

Not all AI suggestions were immediately correct or complete. For example, the initial implementation of the training loop did not clearly separate actor and critic updates, which I had to correct after testing the code. I verified correctness by checking learning curves and confirming that baseline reduced variance as expected. This showed that AI guidance needs to be tested rather than accepted directly.

4. **Learning Reflection**

Through this process, I learned that REINFORCE has inherently high variance due to Monte Carlo return estimation, and that introducing a baseline significantly stabilizes training. I also learned the importance of correctly handling gradient flow in PyTorch

using .detach(). In terms of AI collaboration, I realized that effective use of AI requires iterative questioning, debugging, and validation rather than relying on a single response.

Speaker Notes

- This lab implements REINFORCE and REINFORCE with baseline in the CartPole-v1 environment using PyTorch
- The goal is to study policy gradient methods and variance reduction techniques in reinforcement learning
- A neural network policy outputs action probabilities using a softmax distribution over logits
- REINFORCE uses Monte Carlo returns, while the baseline method reduces variance using a learned value function
- Key result: baseline significantly stabilizes learning and reduces variance across training runs
- Main challenge: debugging gradient flow in PyTorch and correctly separating actor and critic updates
- Insight: variance reduction is critical in policy gradient methods and directly improves training stability